message-relay — Product Requirements Document (PRD)

> **Version:** 1.0 | **Status:** Draft | **Author:** Prashant Verma

> **Related BRD:** [message-relay-BRD.md](../brd/message-relay-BRD.md) | **Domain:** Platform Engineering · Messaging & Delivery

---

Table of Contents

---

1. Product Overview

message-relay is the notification delivery engine that subscribes to Kafka topics published by **rate-sentinel**. It validates incoming message payloads, routes them to the correct vendor channel, tracks every delivery attempt in a durable MongoDB audit store, indexes records in Elasticsearch for fast search, and ensures no message is ever silently dropped — through automatic retry, circuit breaking, dead-letter queuing, and real-time Sentry alerting.

It is the single system of record for all customer notification history across SMS, Email, and WhatsApp channels.

---

2. Goals & Non-Goals

Goals

• Consume all notification events from Kafka with at-least-once delivery guarantee

• Validate payload integrity and mask PII before any storage write

• Deliver messages via pluggable, vendor-agnostic sender implementations

• Recover automatically from transient vendor failures using retry and circuit breaking

• Persist a complete, append-only audit trail of every delivery lifecycle to MongoDB

• Enable sub-100ms search across delivery records via Elasticsearch

• Alert operations in real time when messages are dead-lettered via Sentry

Non-Goals

• Publishing notification events to Kafka (owned by rate-sentinel)

• Rendering message templates or personalising content

• Scheduling future or deferred message delivery (v1.1)

• Confirming end-customer receipt (delivery webhook — v1.1)

• Authentication / authorisation of API callers (open in v1.0; secured in v1.1)

• Multi-region active-active deployment (v1.0)

---

3. User Stories

| ID | As a… | I want to… | So that… |

|---|---|---|---|

| US-01 | Customer support agent | Search deliveries by phone number or email | I can confirm a customer received their OTP within seconds |

| US-02 | Platform operator | See DLQ depth and delivery success rate in Grafana | I know when the system is healthy without querying MongoDB |

| US-03 | Platform operator | Receive a Sentry alert when a message is dead-lettered | I can investigate and replay failed messages before the customer escalates |

| US-04 | Business owner | Query overall delivery success rate | I can report on SLA compliance to stakeholders |

| US-05 | Platform operator | See the full status history of any notification | I can diagnose exactly where and why a delivery failed |

| US-06 | Security officer | Confirm phone numbers and emails are masked in all stored data | I can evidence PII compliance for data protection audits |

| US-07 | Developer | Swap SMS vendors without changing any consumer code | I can reduce vendor costs or improve reliability with minimal risk |

| US-08 | Platform operator | Trust that a Kafka rebalance or restart never loses a message | I can deploy confidently without worrying about silent data loss |

---

4. Functional Requirements

FR-1 Kafka Message Consumption

FR-1.1 — Four independent topic consumers

Priority: P0 — Must Have

Four separate `@KafkaListener` components must consume from dedicated topics:

Consumer Class	Topic	Group ID	Concurrency
`SmsConsumer`	`topic.sms`	`delivery-engine-group`	3 (matches partition count)
`EmailConsumer`	`topic.email`	`delivery-engine-group`	3
`WhatsAppConsumer`	`topic.whatsapp`	`delivery-engine-group`	3
`PaymentConsumer`	`topic.payment`	`delivery-engine-group`	3

All consumers share the same group ID so Kafka distributes partitions across them collectively.

FR-1.2 — Manual acknowledgement (at-least-once delivery)

Priority: P0 — Must Have

All consumers must be configured with `AckMode.MANUAL`. The acknowledgement sequence must be:

```
Consumer receives message
        ↓
Call DeliveryService.process(event)
        ↓
SUCCESS → ack.acknowledge() → Kafka offset committed
EXCEPTION → do NOT ack → message redelivered on restart
```

`ack.acknowledge()` must only be called after `DeliveryService.process()` returns without exception. Exceptions must be caught, logged at ERROR level with `eventId`, and the message left unacknowledged for redelivery.

FR-1.3 — Consumer configuration

Priority: P0 — Must Have

Kafka consumer factory must be configured with:

Property	Value	Reason
`auto.offset.reset`	`earliest`	Process all messages from beginning on new group
`enable.auto.commit`	`false`	Manual ack mode requires this
`max.poll.records`	`100`	Batch size per poll cycle
`session.timeout.ms`	`30000`	Allow 30s processing per record before rebalance
`spring.json.trusted.packages`	`*`	Trust deserialization of NotificationEvent
`spring.json.use.type.headers`	`false`	Use target type directly, not Kafka type header

FR-2 Payload Validation & PII Masking

FR-2.1 — Schema validation

Priority: P0 — Must Have

`PayloadValidator.validate()` must check the following for every event:

Field	Rule
`eventId`	Not null, not blank
`clientId`	Not null, not blank
`recipient`	Not null, not blank
`channel`	Not null, valid enum value
`priority`	Not null, valid enum value
`templateParams`	If present, size ≤ 20 entries

Returns `ValidationResult(valid: boolean, errors: List)`.

FR-2.2 — Recipient format validation

Priority: P0 — Must Have

Recipient format must be validated against the declared channel:

Channel	Valid Format	Regex
`SMS`	E.164 phone number	`^\+?[1-9]\d{6,14}$`
`WHATSAPP`	E.164 phone number	`^\+?[1-9]\d{6,14}$`
`EMAIL`	RFC 5322 email	`^[\w._%+\-]+@[\w.\-]+\.[a-zA-Z]{2,}$`

Format mismatch adds a descriptive error to `ValidationResult.errors`.

FR-2.3 — PII masking before storage

Priority: P0 — Must Have

`PayloadValidator.maskPii()` must mask recipient values before any write to MongoDB, Elasticsearch, or any log statement:

Channel	Input	Masked Output	Rule
`SMS` / `WHATSAPP`	`+911234567890`	`+91****7890`	Keep country code prefix + last 4 digits
`EMAIL`	`prashant@example.com`	`pr**@example.com`	Keep first 2 chars + mask local part

Masking must be applied in `DeliveryService` before `DeliveryRecord` is constructed. The raw recipient is stored separately in `recipientRaw` on the MongoDB document and must never be returned by any API endpoint.

FR-2.4 — Invalid payload routing

Priority: P0 — Must Have

If `ValidationResult.valid == false`:

1. Log at WARN level with `eventId` and errors list

2. Call `DlqHandler.sendToDlq(event, "VALIDATION_FAILED: " + errors)`

3. Call `ack.acknowledge()` — do NOT leave unacknowledged (invalid messages never become valid on retry)

4. Return without creating any `DeliveryRecord`

FR-3 Message Delivery

FR-3.1 — MessageSender interface and Factory pattern

Priority: P0 — Must Have

The `MessageSender` interface defines the delivery contract:

```
public interface MessageSender {
    String send(NotificationEvent event, DeliveryRecord record); // returns vendorMessageId
    NotificationEvent.Channel supportedChannel();
}
```

Three implementations must be provided:

| Class | Channel | Vendor (v1.0) | Replace With |

|---|---|---|---|

| `SmsSender` | `SMS` | Mock (logs + sleep 50ms) | Twilio / Gupshup SDK |

| `EmailSender` | `EMAIL` | Mock (logs + sleep 60ms) | SendGrid / AWS SES SDK |

| `WhatsAppSender` | `WHATSAPP` | Mock (logs + sleep 70ms) | WhatsApp Business API |

`MessageSenderFactory` resolves the correct implementation at startup using Spring's `List` injection mapped by `supportedChannel()`. Unknown channels throw `IllegalArgumentException`.

FR-3.2 — Delivery orchestration

Priority: P0 — Must Have

`DeliveryService.process()` must execute in this sequence:

```
1. Validate payload (PayloadValidator)
       ↓ invalid → DLQ (no record created)
       ↓ valid
2. Mask PII (maskPii)
3. Create DeliveryRecord (status=PENDING, save to MongoDB)
4. Call attemptDelivery() [wrapped by @CircuitBreaker + @Retry]
       ↓ success
5. Set vendorMessageId, deliveredAt, status=SENT
6. Save DeliveryRecord
7. Index to Elasticsearch (async, failures non-fatal)
8. Increment delivery.success counter
       ↓ failure (after retry exhaustion)
5. Call deliveryFallback()
6. Set failureReason, status=DEAD_LETTERED
7. Save DeliveryRecord
8. Index to Elasticsearch
9. Increment delivery.failure counter
10. Send to DLQ + Sentry
```

FR-3.3 — Vendor message ID capture

Priority: P1 — Should Have

On successful delivery, the vendor-returned message identifier (e.g. Twilio SID, SendGrid message ID) must be stored in `DeliveryRecord.vendorMessageId`. This enables future delivery receipt reconciliation when webhook support is added in v1.1.

FR-4 Retry & Fault Tolerance

FR-4.1 — Exponential backoff retry

Priority: P0 — Must Have

Resilience4j `@Retry` must be configured on `attemptDelivery()`:

Property	Value
`maxAttempts`	3
`waitDuration`	2 seconds
`exponentialBackoffMultiplier`	2
Effective retry schedule	Attempt 1 → wait 2s → Attempt 2 → wait 4s → Attempt 3 → fallback

Retry fires on any `RuntimeException` thrown by `MessageSender.send()`. After all attempts fail, control passes to `deliveryFallback()`.

FR-4.2 — Circuit breaker per sender

Priority: P0 — Must Have

Resilience4j `@CircuitBreaker` must wrap `attemptDelivery()`:

Property	Value
`slidingWindowSize`	10 calls
`failureRateThreshold`	50%
`waitDurationInOpenState`	30 seconds
`permittedNumberOfCallsInHalfOpenState`	3

State transitions:

• `CLOSED` → `OPEN` when 5 of last 10 calls fail

• `OPEN` → `HALF_OPEN` after 30 seconds

• `HALF_OPEN` → `CLOSED` if 3 probe calls succeed; → `OPEN` if they fail

When circuit is `OPEN`, `deliveryFallback()` is called immediately without attempting vendor call.

FR-5 Dead Letter Queue & Alerting

FR-5.1 — DLQ Kafka publish

Priority: P0 — Must Have

`DlqHandler.sendToDlq()` must publish the original `NotificationEvent` to `topic.dlq` using `KafkaTemplate`. The publish is async (`CompletableFuture`). Publish failure must be logged at ERROR but must not block the fallback method from completing.

FR-5.2 — Sentry error alert

Priority: P0 — Must Have

After every DLQ publish, `DlqHandler` must capture a Sentry event with:

| Sentry Field | Value |

|---|---|

| Level | `ERROR` |

| Message | `"DLQ: Message delivery failed [{channel}] eventId={id} reason={reason}"` |

| Tag: `eventId` | Original event UUID |

| Tag: `channel` | `SMS`, `EMAIL`, or `WHATSAPP` |

| Tag: `clientId` | Originating client |

| Extra: `recipient` | **Masked value only** |

| Extra: `templateId` | Template identifier |

| Extra: `correlationId` | Payment ID or OTP ID |

| Extra: `reason` | Human-readable failure description |

Sentry alert failure (network/quota issue) must be caught and logged — it must never propagate and break the DLQ handler.

FR-5.3 — DLQ monitor consumer

Priority: P1 — Should Have

A separate `DlqConsumer` must subscribe to `topic.dlq` using consumer group `dlq-monitor-group`:

• Log every DLQ message at ERROR level with `eventId`, `channel`, `clientId`, `partition`, and `offset`

• Call `ack.acknowledge()` after logging — do not reprocess

• This consumer provides a monitor view; it does not re-deliver messages

The `dlq-monitor-group` must be completely independent of `delivery-engine-group` — its lag must not affect delivery consumer metrics.

FR-6 Audit Storage — MongoDB

FR-6.1 — DeliveryRecord document structure

Priority: P0 — Must Have

Every processed event must produce a MongoDB document in the `delivery_records` collection:

| Field | Type | Indexed | Notes |

|---|---|---|---|

| `_id` | ObjectId | PK | Auto-generated |

| `eventId` | String | Yes (unique) | For O(1) lookup |

| `clientId` | String | Yes | For client-level queries |

| `recipient` | String | Yes | Masked value only |

| `recipientRaw` | String | No | Unmasked; not exposed by API |

| `channel` | Enum | No | `SMS`, `EMAIL`, `WHATSAPP` |

| `templateId` | String | No | |

| `templateParams` | Map | No | |

| `status` | Enum | Yes | Current delivery status |

| `statusHistory` | Array | No | Append-only transition log |

| `attemptCount` | Integer | No | Incremented per retry |

| `failureReason` | String | No | Null on success |

| `vendorMessageId` | String | No | From vendor on success |

| `correlationId` | String | No | Payment ID or OTP ID |

| `createdAt` | DateTime | No | Immutable |

| `deliveredAt` | DateTime | No | Set on `SENT` |

FR-6.2 — StatusHistory entry structure

Priority: P0 — Must Have

Every state transition must append an entry to `statusHistory`:

```
{
  "from": "SENDING",
  "to": "SENT",
  "reason": "Vendor ACK: SMS-ABC123",
  "at": "2025-01-01T10:00:00"
}
```

`from` is null for the initial `PENDING` entry. The array is never modified — only appended to.

FR-6.3 — MongoDB repository queries

Priority: P0 — Must Have

`DeliveryRecordRepository` must support:

| Method | Use Case |

|---|---|

| `findByEventId(eventId)` | Idempotency check + detail lookup |

| `findByClientIdAndStatus(clientId, status)` | Client delivery report |

| `countByStatus(status)` | Stats endpoint |

| `findRecentDeadLettered(since)` | DLQ monitoring |

FR-7 Search Index — Elasticsearch

FR-7.1 — DeliveryDocument index structure

Priority: P0 — Must Have

After each status change, a flattened `DeliveryDocument` must be written to the `delivery_records` Elasticsearch index:

Field	ES Type	Notes
`id`	keyword	Maps to MongoDB `_id`
`eventId`	keyword	Exact match search
`clientId`	keyword	Exact match filter
`recipient`	keyword	Masked value; exact match
`channel`	keyword	`SMS`, `EMAIL`, `WHATSAPP`
`templateId`	keyword	
`status`	keyword	`SENT`, `FAILED`, `DEAD_LETTERED` etc.
`attemptCount`	integer	
`failureReason`	text	Full-text searchable
`correlationId`	keyword	
`createdAt`	date	
`deliveredAt`	date	

FR-7.2 — Non-blocking index sync

Priority: P0 — Must Have

Elasticsearch indexing must be fire-and-forget from the perspective of the delivery flow:

```
try {
    searchRepository.save(doc);
} catch (Exception e) {
    log.warn("ES index failed for eventId={}: {}", record.getEventId(), e.getMessage());
    // delivery flow continues unaffected
}
```

An Elasticsearch outage must never cause consumer lag, delivery failures, or circuit breaker trips.

FR-8 Search & Stats API

FR-8.1 — Delivery search endpoint

Priority: P0 — Must Have

```
GET /api/messages/search
```

Query parameters:

Param	Type	Description
`recipient`	String	Masked phone or email

| `status` | String | `SENT`, `FAILED`, `DEAD_LETTERED`, `PENDING` |

| `channel` | String | `SMS`, `EMAIL`, `WHATSAPP` |

| `clientId` | String | Originating client |

| `page` | Integer | Default: 0 |

| `size` | Integer | Default: 20 |

Resolution logic:

```
clientId + status → findByClientIdAndStatus()
channel  + status → findByChannelAndStatus()
recipient         → findByRecipient()
clientId          → findByClientId()
status            → findByStatus()
(none)            → findAll()
```

Results sorted by `createdAt` DESC. Returns Spring `Page`.

FR-8.2 — Delivery record detail endpoint

Priority: P0 — Must Have

```
GET /api/messages/{eventId}
```

Returns the full `DeliveryRecord` from MongoDB including the complete `statusHistory` array. Returns HTTP 404 if no record exists for the given `eventId`.

FR-8.3 — DLQ stats endpoint

Priority: P1 — Should Have

```
GET /api/messages/stats/dlq
```

Returns a JSON summary from MongoDB aggregation:

```
{
  "total": 15420,
  "sent": 15398,
  "deadLettered": 8,
  "pending": 14,
  "successRatePct": "99.95"
}
```

All counts are live queries against MongoDB. `successRatePct` is calculated as `(sent / total) * 100`, formatted to 2 decimal places.

FR-9 Observability & Metrics

FR-9.1 — Prometheus delivery metrics

Priority: P1 — Should Have

`/actuator/prometheus` must export:

| Metric | Type | Description |

|---|---|---|

| `delivery.success` | Counter | Messages successfully delivered |

| `delivery.failure` | Counter | Messages dead-lettered after retry exhaustion |

| `delivery.validation.failed` | Counter | Messages rejected at validation |

| `delivery.retry.attempt` | Counter | Individual retry attempts |

| `delivery.circuit.open` | Gauge | 1 when circuit breaker is open, 0 when closed |

| `kafka.consumer.lag` | Gauge | Per-topic consumer lag (Spring Kafka auto) |

FR-9.2 — Health probes

Priority: P1 — Should Have

`/actuator/health` must report:

• `UP` when Kafka, MongoDB, and Elasticsearch are all reachable

• `DEGRADED` if Elasticsearch is down but Kafka and MongoDB are healthy (delivery continues)

• `DOWN` if Kafka or MongoDB is unreachable

`/actuator/health/readiness` used by Kubernetes readiness probe.

---

5. Non-Functional Requirements

| Category | Requirement | Target |

|---|---|---|

| Performance | Message delivery throughput | > 3,500 msg/sec (mixed channels) |

| Performance | Payload validation throughput | > 25,000 msg/sec |

| Performance | ES search latency | p95 < 100ms |

| Performance | MongoDB write latency | p99 < 12ms |

| Reliability | Delivery success rate | ≥ 99.9% over 30-day rolling window |

| Reliability | Consumer lag (steady state) | < 50 messages |

| Reliability | DLQ alert latency | < 5 seconds from failure to Sentry event |

| Scalability | Horizontal scaling | Stateless JVM; all state in Kafka/MongoDB/Elasticsearch |

| Security | PII masking | No unmasked phone/email in any log, API, or index |

| Security | SAST | Zero Checkmarx critical/high findings on merge |

| Security | Code coverage | SonarCloud gate: ≥ 70% line coverage |

| Operability | Local setup | `docker-compose up -d` + `./mvnw spring-boot:run` < 3 minutes |

| Operability | Crash recovery | All unacknowledged messages redelivered on restart |

---

6. API Contract Summary

```
GET  /api/messages/search             → Page<DeliveryDocument>  (Elasticsearch)
GET  /api/messages/{eventId}          → DeliveryRecord          (MongoDB)
GET  /api/messages/stats/dlq          → DLQ health summary      (MongoDB aggregation)

GET  /actuator/health                 → Health status
```

```
GET  /actuator/health/readiness        → Readiness status
GET  /actuator/prometheus              → Prometheus metrics
```

---

7. Data Models

DeliveryRecord (MongoDB — `delivery_records`)

```
_id               ObjectId PK (auto)
eventId           String   (indexed, unique)
clientId          String   (indexed)
recipient         String   (indexed, masked)
recipientRaw      String   (not indexed, not in API)
channel           String
templateId        String
templateParams    Map<String, String>
status            String
statusHistory     Array<StatusTransition>
attemptCount      Integer
failureReason     String
vendorMessageId   String
correlationId     String
createdAt         DateTime
deliveredAt       DateTime
```

StatusTransition (embedded in DeliveryRecord)

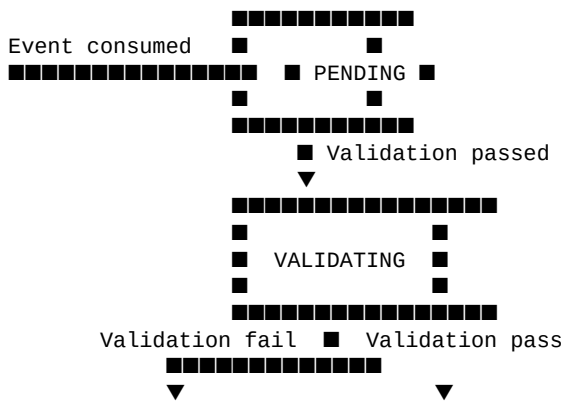
```
from     String (nullable for initial entry)
to       String
reason   String
at       DateTime
```

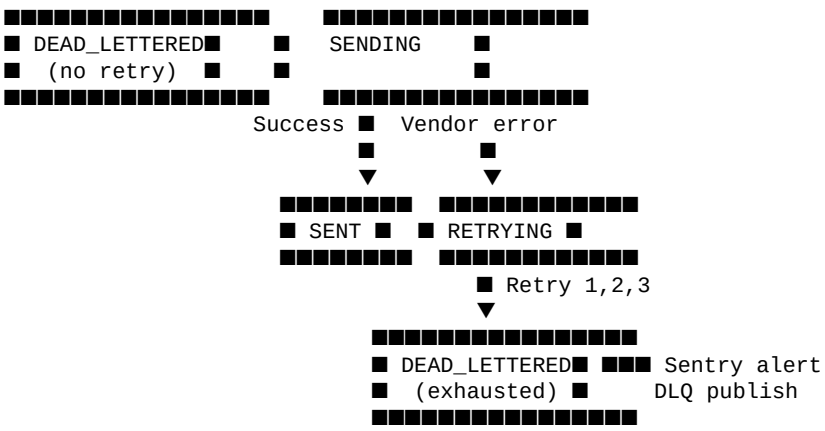
DeliveryDocument (Elasticsearch — `delivery_records`)

```
id             keyword
eventId        keyword
clientId       keyword
recipient      keyword  (masked)
channel        keyword
templateId     keyword
status         keyword
attemptCount   integer
failureReason  text
correlationId  keyword
createdAt      date
deliveredAt    date
```

---

8. Delivery State Machine





Terminal states: `SENT`, `DEAD_LETTERED` — no further transitions.

---

9. Error Handling Standards

| Scenario | Action | Logging |

|---|---|---|

| Kafka deserialization failure | Skip message, log ERROR with raw bytes | ERROR |

| Payload validation failure | Route to DLQ immediately, ack | WARN with errors list |

| Vendor call exception (retryable) | Retry with backoff | WARN per attempt |

| Retry exhausted | Route to DLQ, Sentry alert | ERROR |

| MongoDB write failure | Retry once; if persistent, log and alert | ERROR |

| Elasticsearch sync failure | Log and skip; delivery continues | WARN |

| Sentry alert failure | Log and skip; DLQ publish already done | WARN |

| Circuit breaker open | Route to fallback immediately | WARN with circuit state |

No exception may bubble up past the consumer's `try/catch` block — unhandled exceptions leave messages unacknowledged, causing infinite redelivery loops.

---

10. Dependencies

| Dependency | Version | Purpose |

|---|---|---|

| Spring Boot | 3.2.x | Application framework |

| Spring Kafka | 3.x | Kafka consumer, producer (DLQ) |

| Spring Data MongoDB | 3.2.x | Delivery record persistence |

| Spring Data Elasticsearch | 5.x | Search index management |

| Resilience4j Spring Boot 3 | 2.1.x | Circuit breaker and retry |

| Sentry Spring Boot Starter Jakarta | 7.3.x | Error alerting |

| Micrometer Prometheus | Latest | Metrics export |

| Lombok | Latest | Boilerplate reduction |

| JaCoCo | 0.8.11 | Code coverage |

Infrastructure:

| Service | Version | Purpose |

|---|---|---|

| Apache Kafka | 7.5.x (Confluent) | Message bus (consumed from rate-sentinel) |

| MongoDB | 7.0 | Primary delivery audit store |

| Elasticsearch | 8.11.x | Search index |

| Kibana | 8.11.x | Optional ES UI for dev |

| Prometheus | Latest | Metrics scraping |

| Grafana | Latest | Dashboards |

---

11. Acceptance Criteria Summary

| FR | Acceptance Criteria |

|---|---|

| FR-1.1 | All 4 consumers active after `docker-compose up`. Each logs on first message receipt. |

| FR-1.2 | Simulated crash mid-processing: message redelivered on restart. No message lost. |

| FR-1.3 | Consumer config verified in `KafkaConsumerConfig` bean: `enable.auto.commit=false`, `AckMode.MANUAL`. |

| FR-2.1 | Valid event passes. Event missing `eventId` fails with descriptive error in result. |

| FR-2.2 | `not-a-phone` → validation error for SMS. `not-an-email` → validation error for EMAIL. |

| FR-2.3 | `+911234567890` → `+91****7890`. `prashant@example.com` → `pr**@example.com`. Raw value in `recipientRaw` only. |

| FR-2.4 | Invalid event → DLQ published, ack called, no `DeliveryRecord` created. Verified in integration test. |

| FR-3.1 | `MessageSenderFactory` resolves correct sender for each channel. Unknown channel throws `IllegalArgumentException`. |

| FR-3.2 | Successful delivery → `DeliveryRecord` status `SENT`, `vendorMessageId` set, `deliveredAt` set. |

| FR-4.1 | Mock vendor throws 3 exceptions → all 3 retries attempted (wait 2s/4s/8s) → fallback called. |

| FR-4.2 | 5 of 10 calls fail → circuit opens → next call goes to fallback immediately without hitting vendor. |

| FR-5.1 | `topic.dlq` receives message after retry exhaustion. Message payload matches original event. |

| FR-5.2 | Sentry SDK `captureMessage` called with `ERROR` level, `eventId` tag, masked recipient in extras. |

| FR-6.1 | After delivery, MongoDB document has all required fields. `statusHistory` has at least 2 entries. |

| FR-6.2 | `statusHistory` array length equals number of status transitions. Entries are never modified, only appended. |

| FR-7.1 | After successful delivery, Elasticsearch document exists with matching `eventId` and `status=SENT`. |

| FR-7.2 | Elasticsearch container stopped → delivery continues, consumer lag does not grow, WARN logged. |

| FR-8.1 | `GET /api/messages/search?status=SENT` returns paginated results sorted by `createdAt` DESC. |

| FR-8.2 | `GET /api/messages/{eventId}` returns full record with `statusHistory`. Non-existent ID → 404. |

| FR-8.3 | `GET /api/messages/stats/dlq` returns JSON with `total`, `sent`, `deadLettered`, `successRatePct`. |

| FR-9.1 | `/actuator/prometheus` contains `delivery_success_total` and `delivery_failure_total` counters. |

---

12. Open Questions

| ID | Question | Owner | Status |

|---|---|---|---|

| OQ-01 | Should the search API require authentication? Currently open in v1.0. | Security Officer | Open |

| OQ-02 | What is the Kafka message retention period for `topic.dlq`? (Default Kafka: 7 days) | Platform Lead | Open |

| OQ-03 | Should Elasticsearch be rebuilt automatically from MongoDB on index corruption? | Platform Lead | Backlog |

| OQ-04 | Is `DEGRADED` health status acceptable for Kubernetes readiness probe or should ES outage prevent traffic? | Platform Lead | Open |

| OQ-05 | What is the acceptable maximum DLQ depth before a P1 incident is raised? | Business Owner | Open |

| OQ-06 | Should DLQ replay be automatic (scheduled retry) or manual-only (REST endpoint)? | Product Owner | Backlog |